

Forecasting Precipitation with AI and Radar Data: National Radar Composite and Nowcasting Models Released in IT4LIA

Module 1 · The National Radar Composite IT-DPC-SRI

IT4LIA AI Factory · 22 September 2026 · 09:00–13:00

Today's agenda

Time	Segment
09:00 – 09:45	Welcome & User Creation, Clone repository and setup
09:45 – 10:00	Break
10:00 – 10:15	The Italian national radar composite IT-DPC-SRI
10:15 – 10:45	Notebook session on the IT-DPC-SRI
10:45 – 11:00	Break
11:00 – 11:15	Zarr: storage and management
11:15 – 11:45	Notebook session on Zarr
11:45 – 12:00	Break

Time	Segment
12:00 – 12:45	Hands-on: cloud-native access via ArcoDataHub
12:45 – 12:00	Wrap-up

Materials

Repository folder
module1-radar-datasets/
notebooks/ 00 • 01 • 02 • 03

PART 1

The Italian national radar composite

What the archive contains and how it is organised.

Why data management comes first

Ingredients

- Increasing availability of high-resolution weather and climate data
- Fast-paced progress of data-intensive AI methods

The needs

- Efficient access — read only what you need
- Efficient compression — many years of data
- Efficient updating — the archive grows every 5 minutes

Goal

One unified, cloud-native dataset used to distribute the data (for example to train, validate and test deep-learning nowcasting models).

How a weather radar observes: the Mt. Macaion radar

- **C-band** single-polarisation Doppler radar (EEC DWSR-2500C)
- **Site:** Mt. Macaion, 1866 m a.s.l. (46°29'18" N, 11°12'38" E), complex Alpine orography
- **Operators:** Meteotrentino and Meteobolzano (Civil Protection, Trento and Bolzano)
- **Cadence:** one volume scan every 5 min (288 per day) since the 2010 digital-receiver upgrade

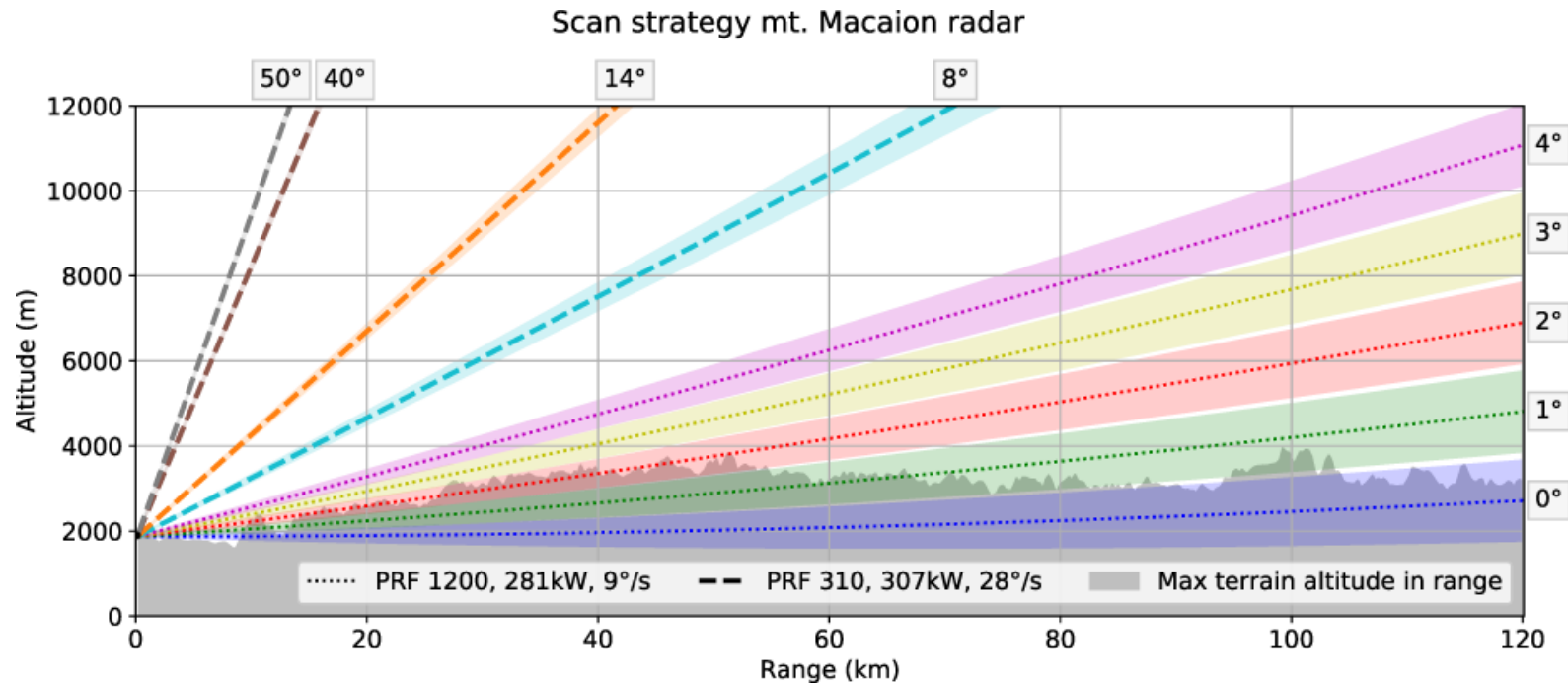
Quality control

Yearly calibration of transmitter and receiver; continuous BITE monitoring; background-noise and solar scans; Doppler filter against fixed-obstacle clutter.

Parameter	Value
Operational range	120 km
Maximum range	480 km
Range resolution	250 m
Beamwidth	0.9°
Wavelength	5.3 cm (5.6 GHz)
Peak power	307 kW
Pulse duration	0.8 μs
Antenna gain	45.8 dB
Ground clutter correction	Doppler filter

Source: Franch et al. (2020), TAASRAD19, Scientific Data 7:234, Table 1.

Scan strategy: a volume of conical sweeps



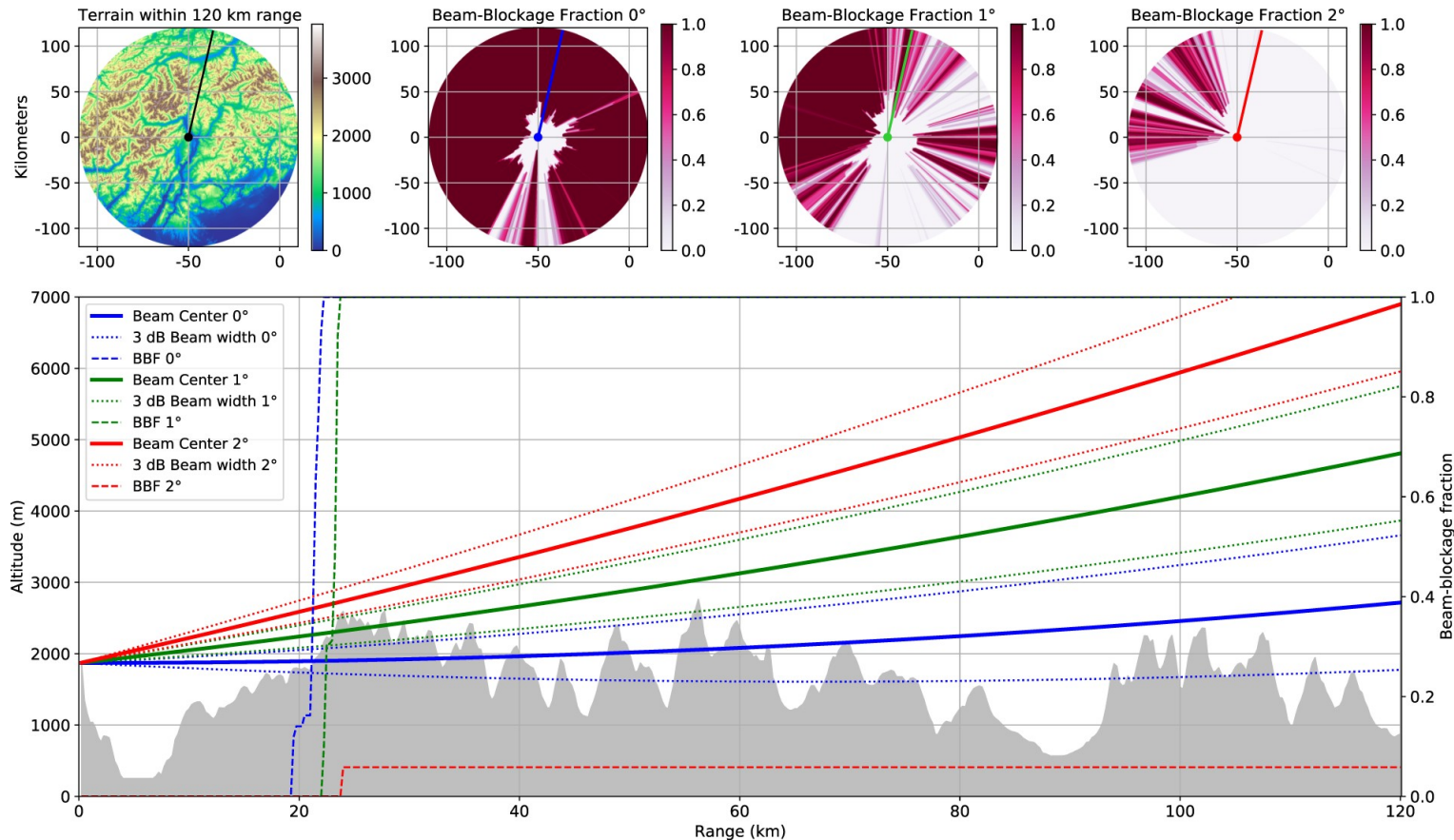
Beam height vs range for each elevation; grey = maximum terrain altitude in each range bin. Source: Franch et al. (2020), TAASRAD19, Scientific Data 7:234, Fig. 1.

Consequence

The lowest sweeps (0°–2°) intersect the terrain: substantial beam blocking (next slide).

- **Every 5 min** the antenna rotates at increasing elevations: 0°–4°, then 8°, 14°, 40°, 50°
- **The beam rises with range:** the 0° beam centre goes from ~1.9 km at the radar to ~2.7 km at 120 km

Beam blockage in complex terrain



- **0°:** blocked almost everywhere beyond the near range
- **1°:** large blocked sectors, mainly to the west
- **2°:** mostly clear; blockage limited to north-western sectors
- **At 13° azimuth:** 0° and 1° fully blocked at ~20–24 km

Why it matters for IT-DPC-SRI

The dataset paper links beam blockage to reduced coverage in some Alpine valleys, and overshooting and partial blocking to radar underestimation along the Alps and Apennines.

Top: terrain and beam-blockage fraction (BBF) at 0°, 1°, 2°. Bottom: cross-section at 13° azimuth; dashed = BBF vs range.
Source: Franch et al. (2020), TAASRAD19, Scientific Data 7:234, Fig. 2.

Source: the Italian national radar network

26

radar systems
20 C-band · 6 X-band

13

operating administrations
(DPC, regions, ENAV, CNMCA)

11

operated by DPC
7 C-band + 4 X-band, dual-pol

Centralised processing

Polar volumes from each radar are ingested at the Centro Funzionale Centrale (CFC) by WIDE (Weather Ingestion Data Engine), which produces the national gridded mosaics.

A gap in the European picture

Italy is not a member of EUMETNET OPERA: pan-European composites have a gap over Italy. IT-DPC-SRI is technically compatible with OPERA products.

Gridded mosaic products

CAPPI

reflectivity at 2–8 km altitude, 1 km step

VMI

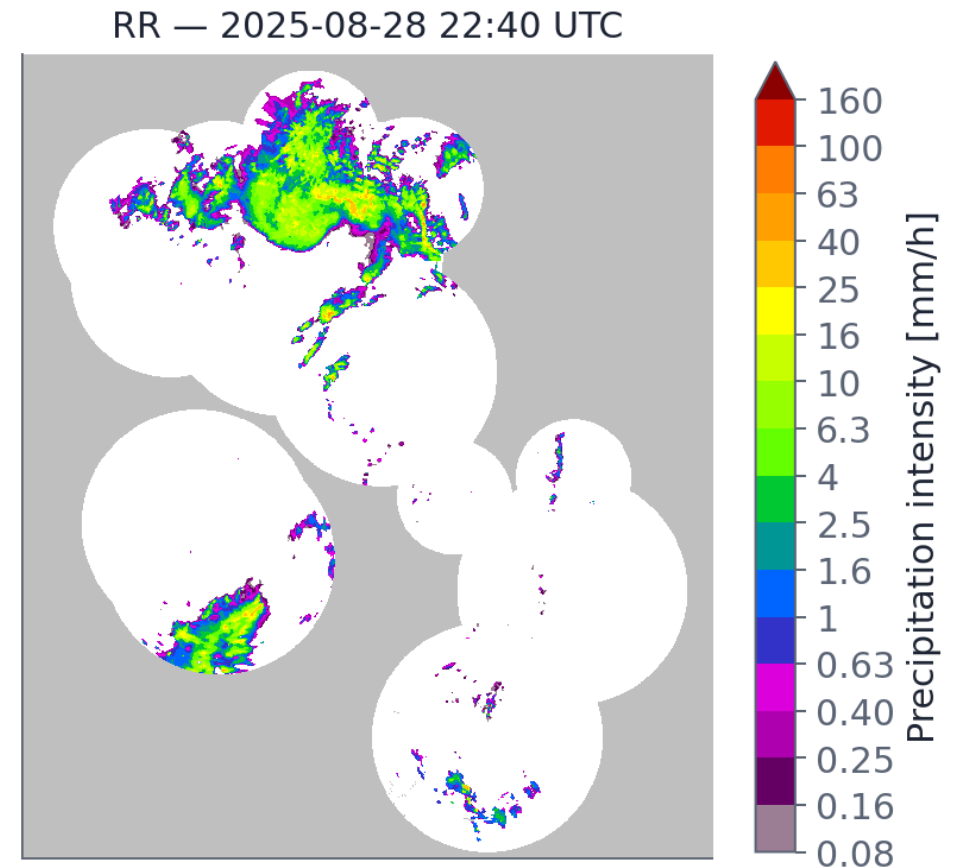
maximum reflectivity in each vertical column

SRI

surface rainfall intensity, mm/h

IT-DPC-SRI: the SRI archive at a glance

- **Product:** DPC Surface Rainfall Intensity, radar-only national mosaic
- **Variable:** RR — instantaneous rain rate, mm/h, float32
- **Grid:** 1400 × 1200 pixels, 1 km, Transverse Mercator
- **Cadence:** 15 min to Jun 2014, 10 min to Jun 2020, 5 min since Jul 2020
- **Period:** 2010–2025 static archive (1 039 785 timesteps); live version appended every 5 min
- **Licence:** CC BY-SA 4.0 — credit DPC (source) and FBK (processing)
- **Access:** Zenodo snapshot (DOI) • ECMWF European Weather Cloud (anonymous) • ArcoDataHub (live, login)



See it in practice → [Notebook 01 • Structure at a glance](#)

What a national rain-rate archive is used for

Nowcasting

0–2 h forecasts from the latest radar frames —
Module 2

Hydrology / Agriculture

Flash-flood warning,
catchment rainfall
accumulation

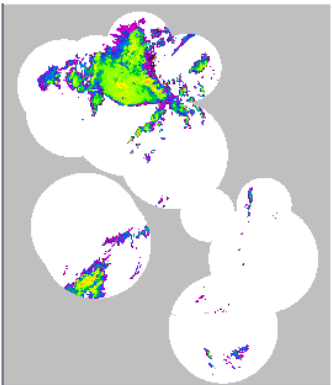
Insurance

Detecting convective
hazards, risk assessment
on historical records

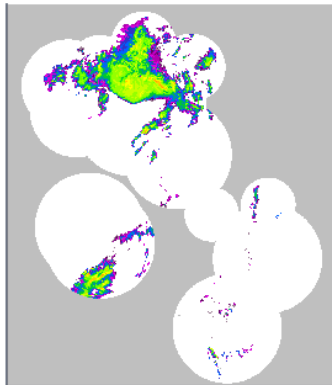
Climatology

“Long-term”
precipitation statistics at
km scale

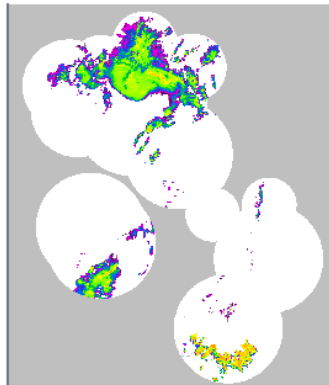
21:55



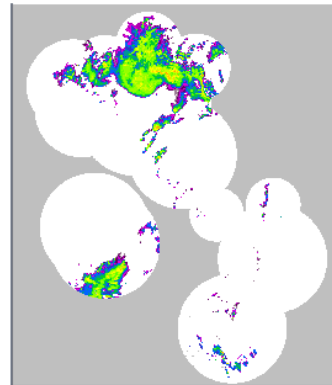
22:10



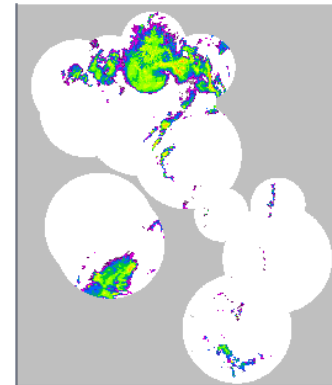
22:25



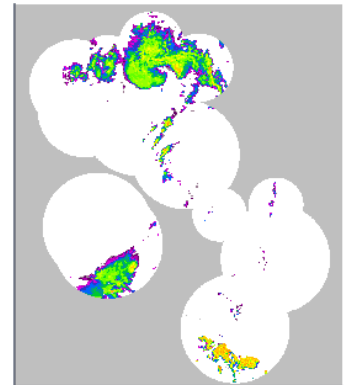
22:40



22:55



23:10



28 August 2025, 21:55 → 23:10 UTC, every 15 minutes (examples/sample_data.nc) • See it in practice → [Notebook 01](#) • [Movie of the RR](#)

From fragmented archives to one datacube

Heterogeneous sources extent, grid spacing and projection also changed over time

OPERA BUFR

≈2010–2020 archive

Binary; specialised decoders; unsuited to array-oriented analysis

HDF5 / ODIM

2020 onward archive

More accessible, but no efficient partial reads over the network

GeoTIFF / FTP

real-time, since ≈2020

File-by-file access to recent data; FTP collection since 2024



Harmonisation pipeline

1 • Spatial regridding

All periods reprojected onto the current operational grid (1200 × 1400, 1 km, Transverse Mercator) with the least destructive transformations; rain-rate units handled consistently.

2 • Temporal alignment

Timestamps snapped to the nearest exact multiple of 5 min UTC. **No temporal interpolation of rain rates.**

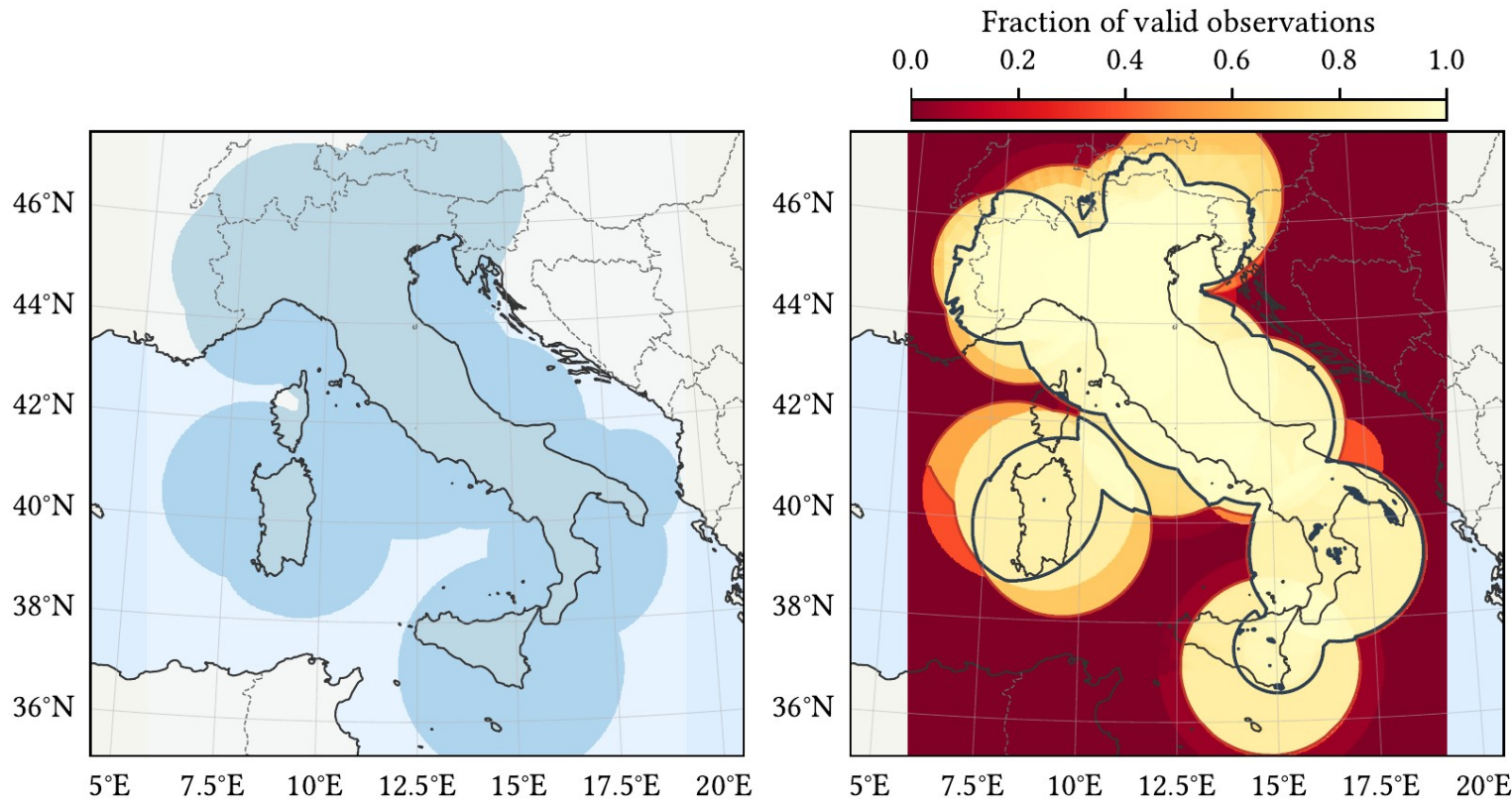
3 • Streamlining

Quality flags and auxiliary fields dropped; minimal filtering beyond the operational chain.

4 • ARCO output

One CF-compliant Zarr datacube; gaps listed in missing_times.

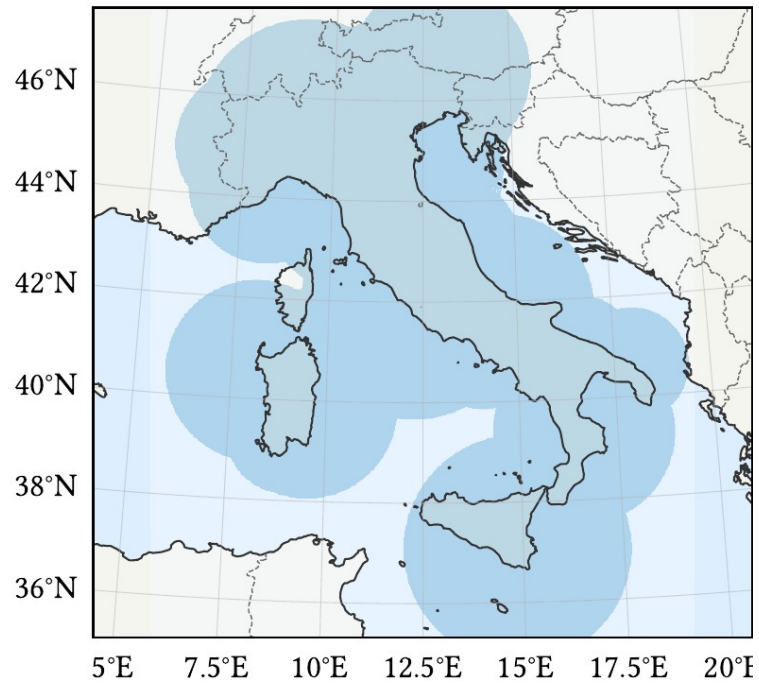
Spatial domain and coverage



Left: radar footprint (cells with ≥ 1 valid observation). Right: fraction of valid observations per cell (10 000 uniformly spaced timesteps; contours at 0.5 and 0.9).

- **Domain** $\approx 35\text{--}48^\circ \text{N}$, $5\text{--}20^\circ \text{E}$; 1400×1200 cells at 1 km
- **Footprint:** peninsula, Sicily, Sardinia, part of the surrounding seas and neighbouring countries
- **> 90 %** valid over the Po Valley and central Italy (overlapping beams)
- **Degrades** over the sea beyond the typical ~ 150 km radar range
- **Reduced** in some Alpine valleys and part of Sicily: beam blockage and missing coverage

Coverage: NaN is not "no rain"

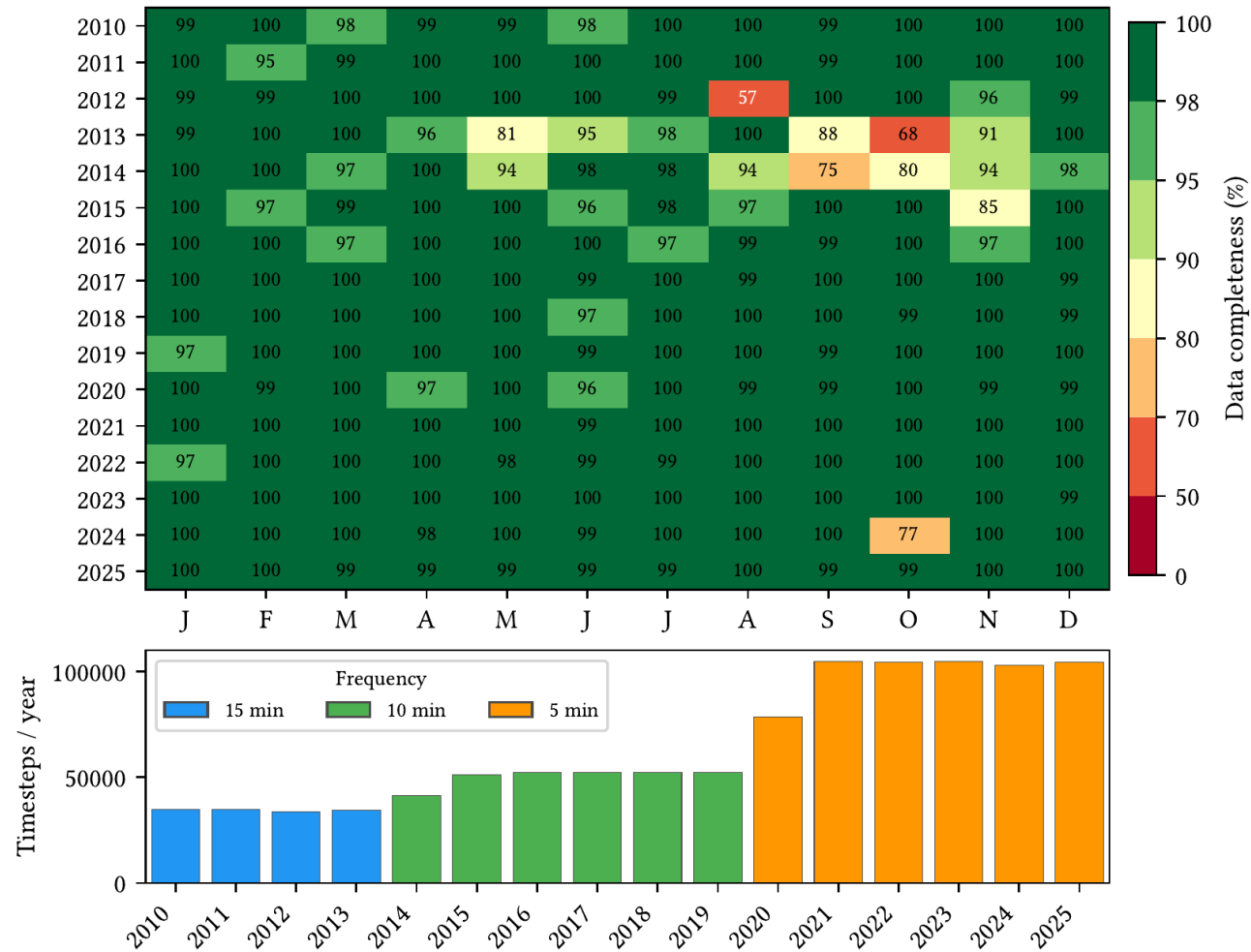


- Coverage can change from one timestep to the next (radar outages)
- 0 mm/h = observed dry; NaN = not observed

Rule of thumb

Mask NaN explicitly in statistics, losses and scores. Module 2's NaN filter and masked loss exist exactly because of this.

Temporal coverage



Top: monthly completeness (%). Bottom: timesteps per year by cadence era.

98.5 %

completeness
(2010–2025 archive)

15 530

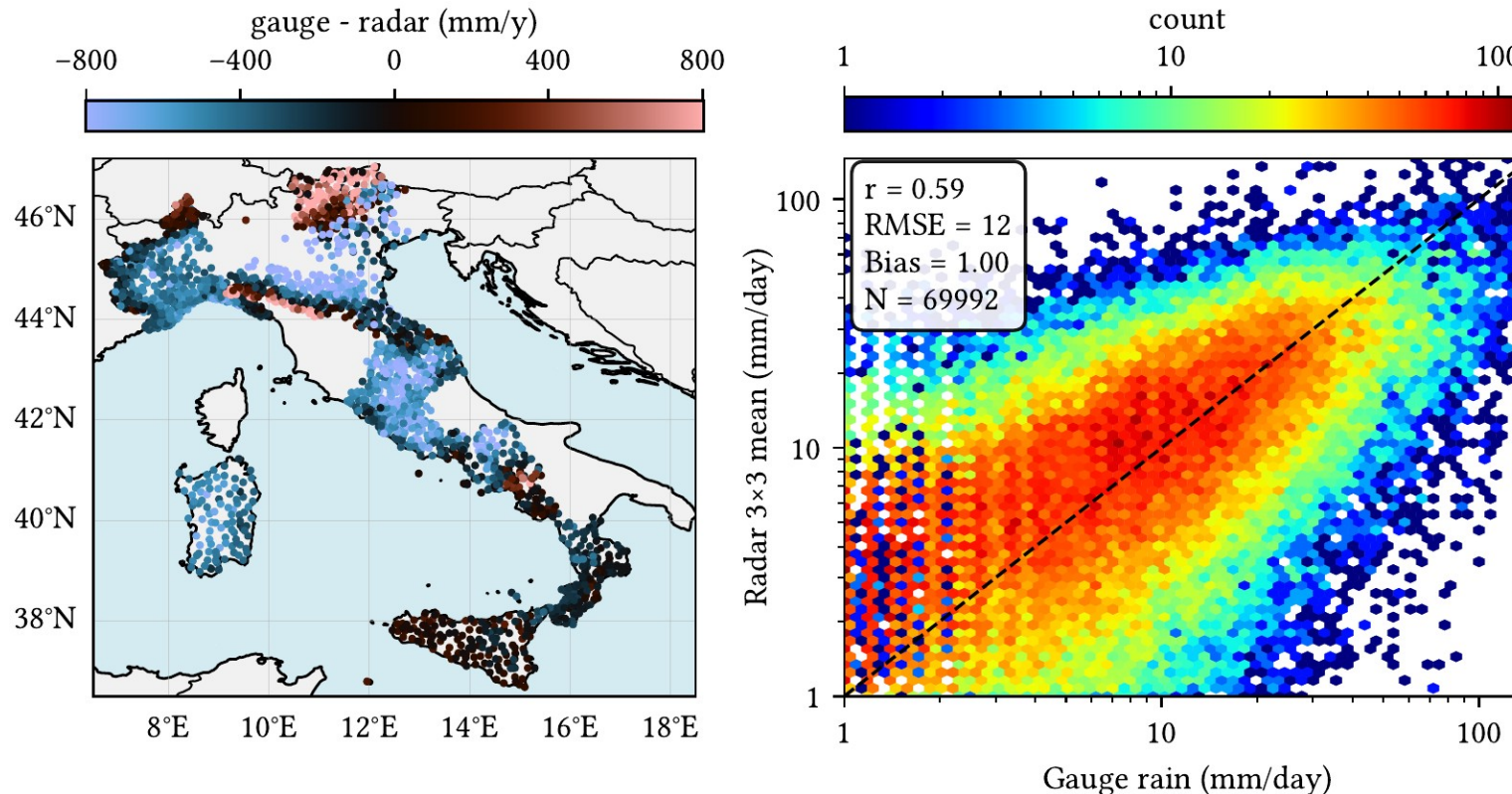
missing timesteps
(of 1 039 785)

- **Three cadence eras:** 15 min (2010 – Jun 2014), 10 min (Jun 2014 – Jun 2020), 5 min (from Jul 2020); the annual count doubles in 2020–21
- **Largest gaps:** Aug 2012 (57 %), Oct 2013 (68 %), Sep 2014 (75 %), Oct 2024 (77 %)

Tip

`missing_times` (static archive) lists every absent timestamp: find gaps without scanning RR.

Test against observations: rain gauges



Left: 2023 gauge - radar difference per station (mm/yr). Right: daily gauge vs radar 3 × 3 km mean (mm/day), station-days ≥ 1 mm/day.

Bias Correction

Left to you! Consider that it depends on the region!

Setup

MeteoNetwork stations, year 2023

Daily totals; radar = 3 × 3 km mean around each gauge

Days < 1 mm removed (0.2 mm bucket)

0.59

Pearson r

N = 69 992 station-days · 5693 stations

12

RMSE (mm/day)

Orographic signal

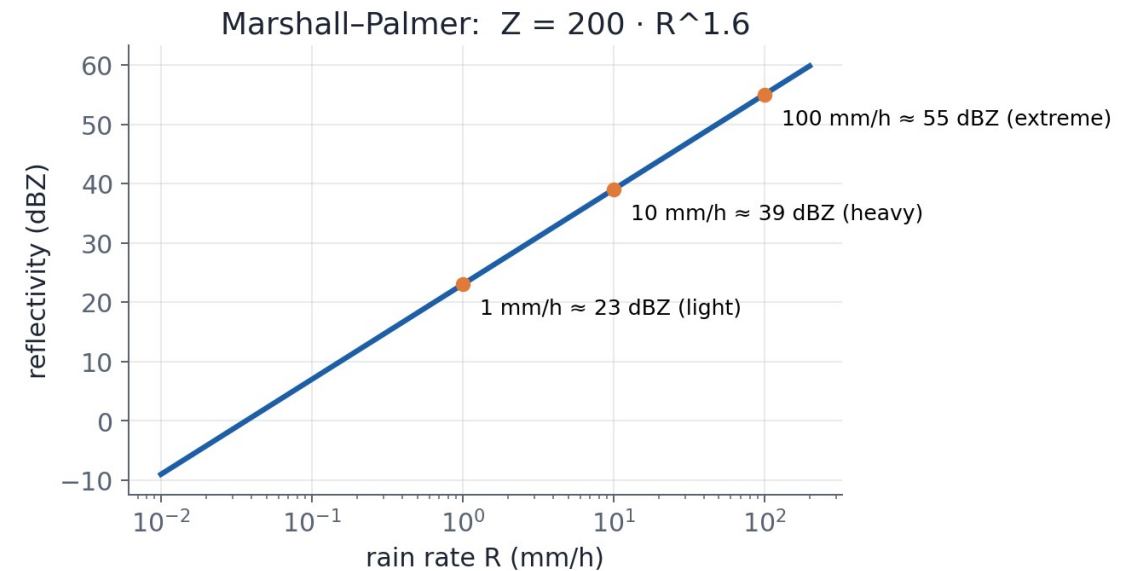
Positive values (radar < gauge) along the Alps and Apennines: beam overshooting and blocking. Mostly negative elsewhere.

From reflectivity to rain rate

- Radar measures **reflectivity Z** of hydrometeors
- Log scale: $\text{dBZ} = 10 \cdot \log_{10} Z$
- A **Z-R relation** converts it to rain rate
 - Marshall-Palmer: $Z = 200 \cdot R^{1.6}$
- RR in the archive is already a rain rate (mm/h) after compositing

Why it matters later

IRENE converts mm/h $\rightarrow$ dBZ, clips to 0-60 dBZ and rescales to [-1, 1] before training.



Real relation, plotted from $Z = 200 \cdot R^{1.6}$

Projection and grid geometry

Parameter	Value
Projection	Transverse Mercator
Latitude of origin	42° N
Central meridian	12.5° E
Scale factor	1.0
False easting / northing	0 m / 0 m
Ellipsoid	WGS 84
Cell size	1 km × 1 km

proj4

```
+proj=tmerc +lat_0=42 +lon_0=12.5 +k=1 +x_0=0  
+y_0=0 +ellps=WGS84 +units=m +no_defs
```

Pixel → projected coordinates (GDAL GeoTransform)

(-600000, 1000, 0, 650000, 0, -1000)

Origin at the upper-left corner: $x = -600$ km, $y = +650$ km; 1000 m pixels, y decreasing downwards.

→ $x \in [-600, +600]$ km (1200 px), $y \in [-750, +650]$ km (1400 px)

CF grid mapping

`crs` is a scalar variable: no data, only attributes defining the projected CRS.

`RR:grid_mapping = "crs"` links the data to its CRS.

`x`, `y` (m) are dimension coordinates; `lat`, `lon` are 2D auxiliary coordinates.

CF-aware tools (xarray, GDAL) interpret the grid automatically for reprojection and overlays.

Dataset structure

Name	Dims	Notes
time	3,564,898	2010 → 2049, pre-allocated
y	1400	metres, descending
x	1200	metres, ascending
lat, lon	(y, x)	2D geographic coords
crs	()	projection metadata
RR	(time, y, x)	float32, rain rate

projection (PROJ4)

```
+proj=tmerc +lat_0=42 +lon_0=12.5 +k=1 +x_0=0 +y_0=0 +ellps=WGS84
```

xarray.Dataset

Dimensions: (y: 1400, x: 1200, time: 3564898)

Coordinates:

time	(time)	datetime64[ns]	2010-01-01 ... 2049-12-31T23:55:00
x	(x)	float64	-5.995e+05 -5.985e+05 ... 5.995e+05
y	(y)	float64	6.495e+05 6.485e+05 ... -7.495e+05

Data variables:

crs	()	float32	...
lat	(y, x)	float64	dask.array<chunks=(1400, 1200), meta=np.nd...
lon	(y, x)	float64	dask.array<chunks=(1400, 1200), meta=np.nd...
RR	(time, y, x)	float32	dask.array<chunks=(1, 1400, 1200), meta=np....

	Array	Chunk
Bytes	21.79 TiB	6.41 MiB
Shape	(3564898, 1400, 1200)	(1, 1400, 1200)
Dask graph	3564898 chunks in 2 graph layers	
Data type	float32 numpy.ndarray	

Indexes: (3)

Attributes:

Author : Gabriele Franch

Conventions :

history : Created at 2025-09-11T11:41:52+02:00

Copyright : Dipartimento Protezione Civile Nazionale rete RADAR

Licence : CC BY-SA 4.0

Processed by : Fondazione Bruno Kessler

title : Italian Radar DPC SRI Archive

zarr_creation : Created on 2025-09-11T11:41:52+02:00 by Gabriele Franch (franch@fbk.eu)

zarr_dataset_ve... 0.1.0

See it in practice → [Notebook 01 • Where the projection info lives \(crs\)](#)

Note on the time-axis

1 • last_valid

The time axis is pre-allocated up to 2050 so new frames can be written in place.

Every step after `ds.attrs["last_valid"]` is NaN.

Check it before choosing a date range.

2 • Non-constant cadence

15-minute steps to mid-2014, 10-minute to June 2020, 5-minute since July 2020.

Never compute a position from the start date and a nominal step.

Always select by label: `ds.sel(time=...)`.

```
last_valid = pd.Timestamp(ds.attrs["last_valid"])  
event = ds["RR"].sel(time=slice("2025-08-28T21:55", "2025-08-28T23:20")) # by label
```

See it in practice → [Notebook 01 • Note on the time-axis](#)

Nootebook Session

User creation, Clone repo, Setup environment

PART 3

Zarr: storage and management

How a 22-TiB array becomes something you can open in a second and slice over HTTP.

Zarr in one slide

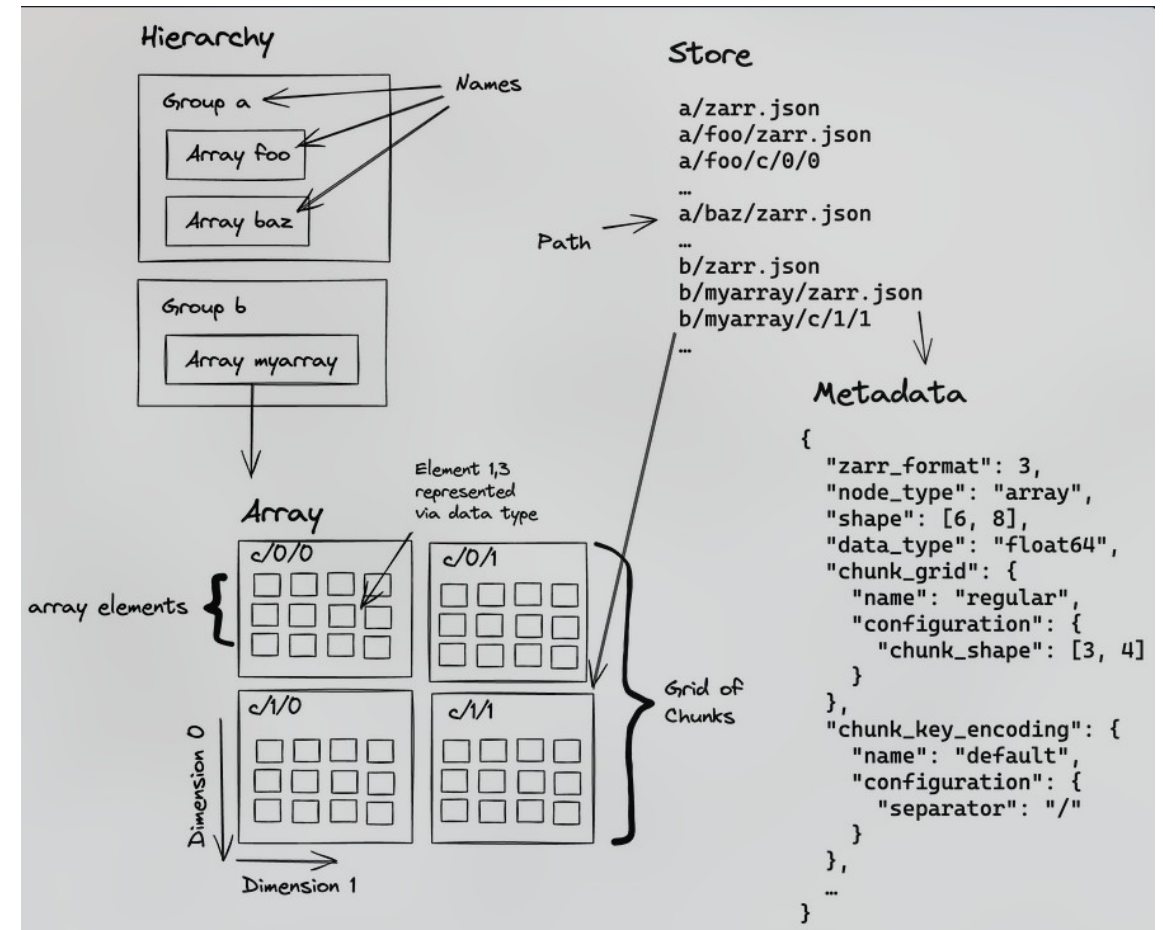
- Open-source, **cloud-native** format for N-dimensional arrays
- Read only the chunks you touch; write chunks in parallel
- Arrays split into **chunks** — each chunk is an independent, compressed object
- Same data model as HDF5/NetCDF: groups, arrays, attributes



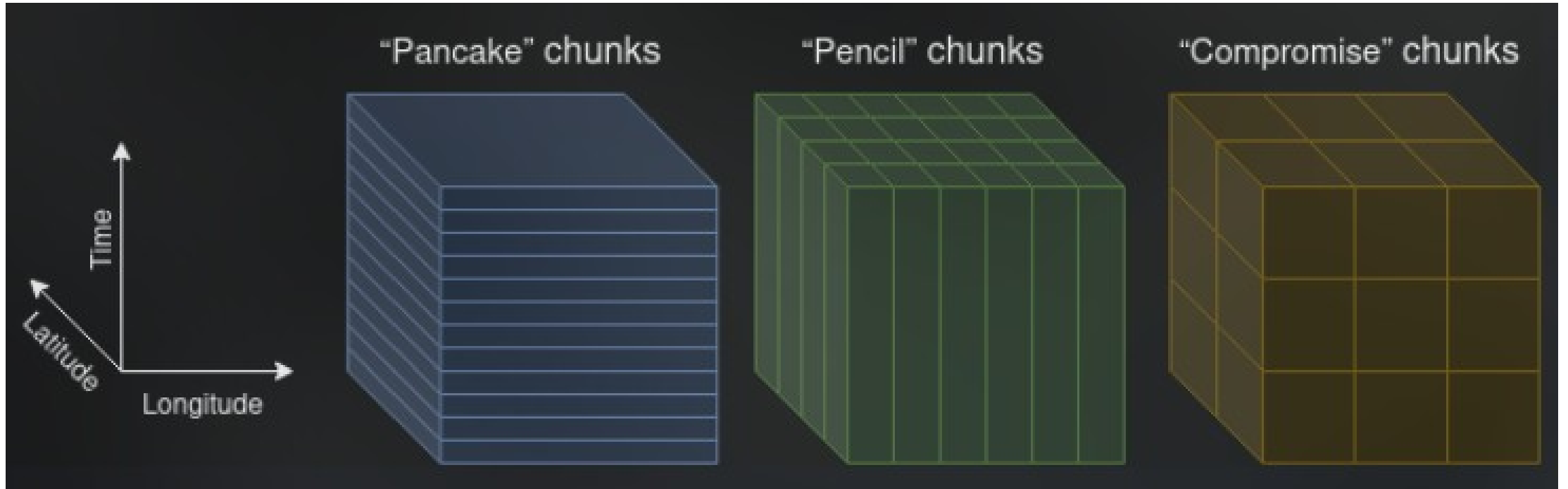
See it in practice → [Notebook 02](#) • [§1 Inspect chunking & compression](#)

Anatomy of a Zarr hierarchy

- **Hierarchy** — the tree, starting at the root
- **Group** — a node that holds child nodes
- **Array** — a leaf that holds data, split into chunks
- **Metadata** — every group and array carries its own attributes (JSON)



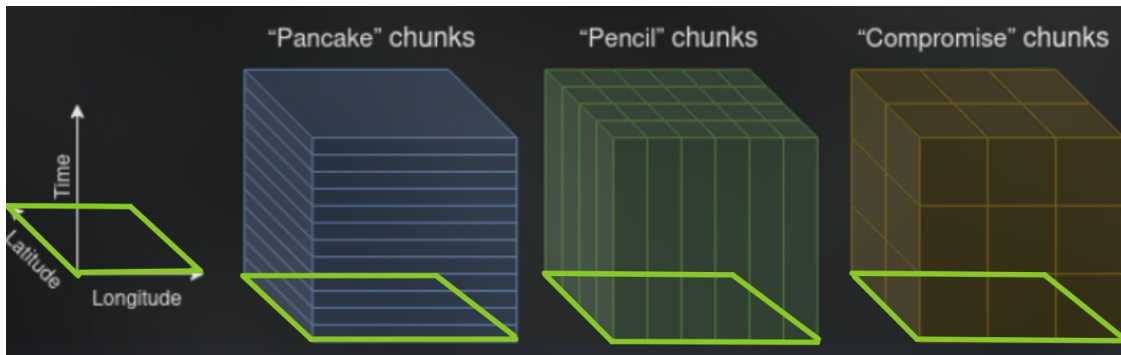
Chunking strategies



The partitioning decides which reads are cheap. There is no universally best choice — only a best choice for your access pattern.

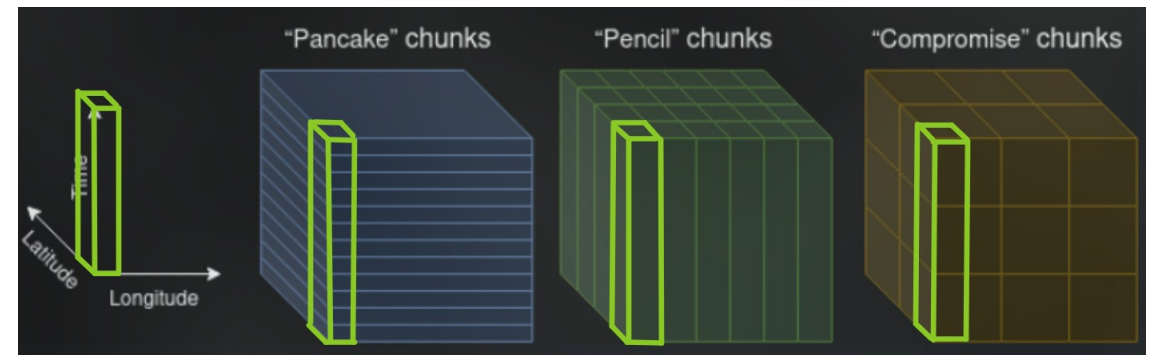
Access pattern → chunks touched

A few time steps, whole domain



Pancakes: touches only the chunks of those steps.

One location over time



Pancakes: touches every time chunk. Pencils: a single chunk.

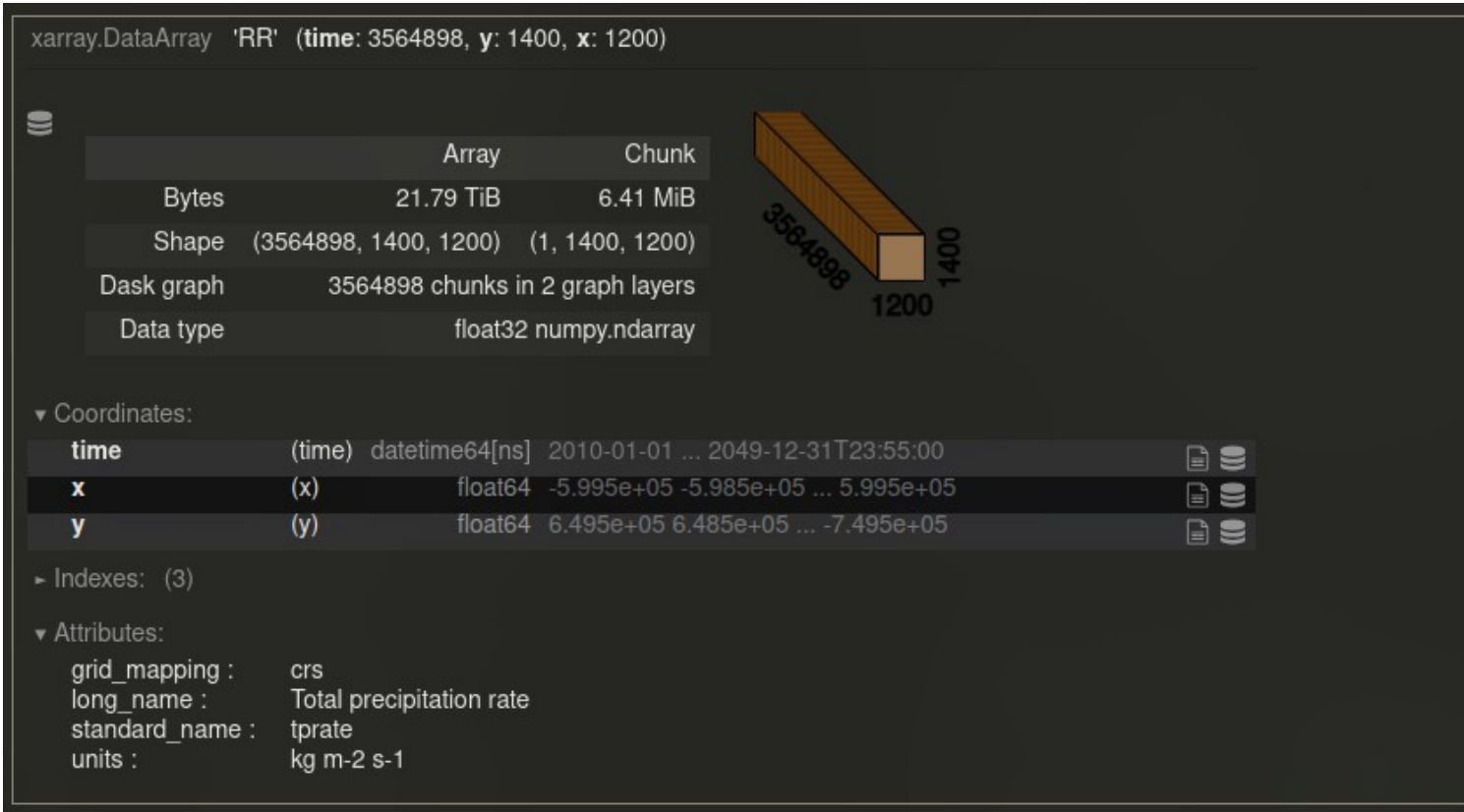
See it in practice → [Notebook 02](#) • [§4 Downloads chunk-by-chunk](#)

Question: which chunking would you choose?

#	Task on the radar archive	Reads per request	Your strategy
1	Animate a storm over all of Italy	$\sim 20 \text{ steps} \times 1400 \times 1200$	
2	Train IRENE on random datacubes	$24 \times 256 \times 256$, millions of times	
3	Rain-gauge comparison at one pixel	$16 \text{ years} \times 1 \times 1$	

Think about: number of chunks touched per request, size of each chunk once decompressed, and how often each task runs.

What the archive does: pancakes



- Chunk = **(1, 1400, 1200)** — one 5-min frame
- **6.41 MiB** per chunk once decompressed
- **Ideal for:** multiple times, whole region

Chosen for

Live appends (one new frame = one new chunk) and full-domain reads.

See it in practice → [Notebook 02 · §1. Inspect the remote store's chunking & compression](#)

The radar archive as a Zarr tree

```

italian-radar-dpc-sri.zarr/
├── time/      (time)      datetime64
├── y/         (y)         metres
├── x/         (x)         metres
├── lat/       (y, x)      float64
├── lon/       (y, x)      float64
├── crs/       ()          proj4, crs_wkt, ...
├── RR/        (time, y, x) float32
│   ├── .zarray
│   └── .zattrs          grid_mapping: "crs", units, ...
├── .zattrs          licence, history, last_valid
├── .zgroup
└── .zmetadata
  
```

- `_ARRAY_DIMENSIONS` attribute tells xarray the dimension names
- `grid_mapping = "crs"` links data to projection (GeoZarr-style)
- Global `last_valid` marks the live data extent
- `.zmetadata` = everything above in one file

Why a 22-TiB array is practical to publish

21.79 TiB uncompressed

55.8 GB on disk · ~400× smaller

- Most pixels are dry: long runs of zeros compress extremely well
- Many NaN (no coverage)

Consolidated metadata

All .zattrs/.zarray files merged into one .zmetadata.

Opening a remote store = one HTTP request, not one per array.

Reading: zarr vs xarray

zarr – positional

```
import zarr

root = zarr.open("store.zarr", mode="r")
print(root.tree())

rr = root["RR"]
block = rr[0:100, :, :]
```

xarray – labelled

```
import xarray as xr

ds = xr.open_zarr("store.zarr")
ds

rr = ds["RR"]
event = rr.sel(time=slice("2025-08-28T21:55",
                          "2025-08-28T23:20"))
```

- **Modes:** "r" read · "a" append/edit · "w" overwrite (careful)

See it in practice → [Notebook 01 · Connecting](#)

Writing and updating

create a subset

```
ds.to_zarr(
    "subset.zarr", mode="w", zarr_format=2,
    encoding={"RR": {"chunks": (1, 1400, 1200),
                       "dtype": "float32"}},
)
```

update in place

```
root = zarr.open("archive.zarr", mode="a")
i = index_of(new_time)          # look it up!
root["RR"][i, :, :] = new_frame # one chunk

# xarray: write only a region
edit.to_zarr("archive.zarr",
             region={"time": slice(i, i + 1)})
```

Pre-allocate, don't append

Resize or pre-allocate the time axis, write by index. That is why this archive's time axis already reaches 2050.

See it in practice → [Notebook 02 · §5 Choosing chunk shape](#)

Dask: lazy computation over chunks

open with chunks= → dask-backed

```
ds = xr.open_dataset(url, engine="zarr",
                    chunks="auto")
rr = ds["RR"]      # dask array, 22 TiB
rr.data.chunksize  # (19, 1400, 1200)
```

build the graph, then run it

```
day = rr.sel(time=slice("2025-08-28T00:00",
                          "2025-08-28T23:55"))
mean = day.mean("time")  # nothing read yet

type(mean.data)          # dask.array.Array

result = mean.compute()  # now it reads
```

A graph, not data

Operations only describe the work. Dask then streams chunks through the reduction, so you can average far more data than fits in RAM.

Memory vs. parallelism

The default threaded scheduler decompresses several chunks at once.
Cap num_workers when peak memory matters more than speed.

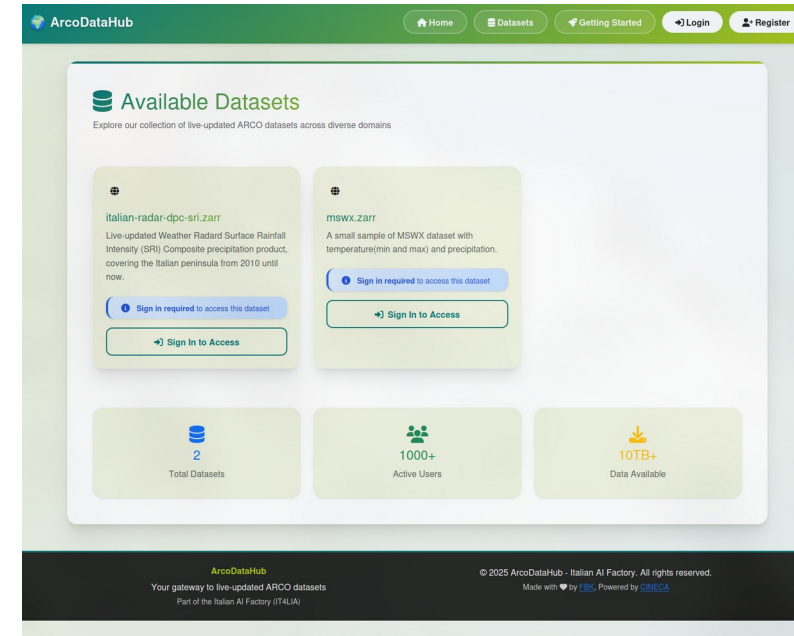
See it in practice → [Notebook 02 · §6 Dask](#)

Analysis-Ready, Cloud-Optimised (ARCO) — and ArcoDataHub

- **Analysis-ready:** community standards, rich metadata
- **Cloud-optimised:** chunked formats, parallel access
- Zarr satisfies both by design



ECMWF: data stores turning ARCO



```
url = f"https://{user}:{key}@api.arcodatahub.com/S3/italian-radar-dpc-sri.zarr"
ds = xr.open_dataset(url, engine="zarr")
# catalogue: catalogue.arcodatahub.com/collections/italian-radar-dpc-sri
```

See it in practice → [Notebook 01 • Connecting](#)

PART 4

Hands-on: cloud-native access

Open the live archive, understand its layout, retrieve one storm, and export a local extract.

Setup (do this first)

```
git clone <course-repository-url> seminars
cd seminars
cp .env.example .env          # paste your ArcoDataHub credentials
cd module1-radar-datasets
uv sync                       # creates .venv from uv.lock
uv run jupyter lab            # then: 00_setup_check.ipynb
```

```
.env (repo root, git-ignored)
ARCODATAHUB_USERNAME=...
ARCODATAHUB_ACCESS_KEY=...
```

One credentials file

.env at the repository root is shared by Module 1 and Module 2. Notebooks find it with `load_dotenv(find_dotenv())`.

If you see an SSL error

Make sure the notebook's first cell (import `aiohttp`) ran before opening the store.

Notebooks

Notebook	Agenda slot	You will
00_setup_check.ipynb	Before you start	Check the kernel, every required package and ArcoDataHub connectivity
01_dataset_structure.ipynb	Radar variables & applications	Open the store lazily, read projection, last_valid and cadence, plot one frame
02_zarr_format.ipynb	Zarr: storage & management	Inspect chunks and codecs, time small vs large reads, write and dissect a local Zarr store
03_arcodatahub_hands_on.ipynb	Hands-on	Retrieve the 28 Aug 2025 storm, animate it, export a NetCDF extract, and describe a regional cube with statistics

Exercises

- 1 • Your own event. Pick a date and window (check `last_valid` first), retrieve it and animate it.
- 2 • Wet vs dry bytes. Time the download of a wet and a dry frame. Same shape — why is one faster?
- 3 • Regional crop. Repeat notebook 03 on a box around your region with `.isel(y=..., x=...)`. How much smaller is the result? Is the download proportionally faster?
- 4 • Chunking in practice. Extract one pixel's time series for one day. Count the chunks touched, then rechunk the extract to pencils and compare.
- 5 • Describe it with numbers. For that regional cube: accumulation map (mm), areal mean and maximum, hyetograph, wet-area fraction, intensity percentiles — with NaN excluded, not counted as dry. Check that mean rate $\times$ duration equals the mean accumulation.

See it in practice → [Notebook 03 • Describe your event with numbers](#)

Summary

Task	zarr	xarray	Choice
Dataset inspection	tree()	labelled repr	
Fast positional reads	✓	✓	
Writing large volumes	✓	✓	
Updating records	index assign	region=	
Datacube manipulation	–	✓	
Rechunking	manual	.chunk()	

Pros

- Community-driven, open specification
- Parallel, chunk-level I/O
- Gentle learning curve via xarray
- Fits a wide variety of needs

Cons

- No computation on its own (use xarray/dask)
- v2 → v3 transition still settling
- Chunking must match access patterns

References and credits

- Franch, G. et al. (2025). IT-DPC-SRI: A Cloud-Optimized Archive of Italian Radar Precipitation (2010–2025). Zenodo, doi:10.5281/zenodo.18637608.
- Giazzi, M. et al. (2022). Meteonetwork: An Open Crowdsourced Weather Data System. Atmosphere, 13(6), 928.
- ArcoDataHub catalogue — catalog.arcodatahub.com/collections/italian-radar-dpc-sri
- Radar data: Dipartimento della Protezione Civile Nazionale (rete RADAR), processed by Fondazione Bruno Kessler — CC BY-SA 4.0
- **Tanks to G. Tomezzoli for the presentation materials (*a gentle introduction to zarr* from AIWCAS25)**